

5. Design of Image thresholding using Xilinx Vivado (Gray to Binary image conversion)

Aim:

To design and simulate a Verilog-based image thresholding system that converts a 256×256 grayscale image into a binary image using a UUT module, with pixel data provided through a text file and the processed output written back to a text file.

Software required

Verilog HDL

Xilinx Vivado

Python (with Pillow and NumPy libraries) to convert image to text file and back

Operating system: Windows

Procedure

A grayscale image is converted into a 256×256 format and stored as a one-dimensional text file containing 65,536 pixel values (0–255), with one pixel per line.

A Verilog UUT is designed to perform gray-to-binary conversion using an internally defined threshold. If the input pixel value is greater than or equal to the threshold, the output is set to 255; otherwise, it is set to 0.

A Verilog testbench is written to open the input text file, read pixel values sequentially, and apply them to the UUT.

The binary output produced by the UUT for each pixel is captured by the testbench and written into an output text file.

The simulation is executed, and after completion, the generated output text file is converted back into an image using Python to visualize the thresholded result.

Python program to convert the 256x256 image to 1D form suitable for verilog module

```
from PIL import Image
import numpy as np

img = Image.open("input_image.png").convert("L")
img = img.resize((256, 256))
img_array = np.array(img, dtype=np.uint8)

with open("image_in.txt", "w") as f:
    for pixel in img_array.flatten():
        f.write(str(int(pixel)) + "\n")
```

Testbench program

```

`timescale 1ns/1ps

module tb_image_threshold;

    parameter WIDTH = 256;
    parameter HEIGHT = 256;
    parameter SIZE = WIDTH * HEIGHT;

    reg [7:0] gray_pixel;
    wire [7:0] binary_pixel;
    reg [7:0] image_mem [0:SIZE-1];

    integer i;

    integer infile, outfile;

    image_threshold_uut UUT (
        .gray_pixel(gray_pixel),
        .binary_pixel(binary_pixel)
    );

    initial begin

        infile = $fopen("image_in.txt", "r");
        outfile = $fopen("image_out.txt", "w");

        for (i = 0; i < SIZE; i = i + 1) begin
            if (!$feof(infile))
                $fscanf(infile, "%d\n", image_mem[i]);
            else begin
                $display("ERROR: Insufficient image data.");
                $finish;
            end
        end

        for (i = 0; i < SIZE; i = i + 1) begin
            gray_pixel = image_mem[i];
            #1;
            $fdisplay(outfile, "%d", binary_pixel);
        end
    end

```

```
$fclose(infile);  
$fclose(outfile);  
$finish;  
end  
endmodule
```

Verilog program for Gray to binary conversion

```
module image_threshold_uut (  
    input wire [7:0] gray_pixel, // grayscale input (0–255)  
    output reg [7:0] binary_pixel // binary output (0 or 255)  
);  
    localparam [7:0] THRESHOLD = 8'd128;  
    always @(*) begin  
        if (gray_pixel >= THRESHOLD)  
            binary_pixel = 8'd255; // white  
        else  
            binary_pixel = 8'd0; // black  
        end  
    end  
endmodule
```

Python program to convert the 1D output data generated by verilog module to 256x256 image

```
import numpy as np  
from PIL import Image  
data = np.loadtxt("image_out.txt", dtype=np.uint8)  
img = data.reshape((256,256))  
Image.fromarray(img).save("binary_output.png")
```

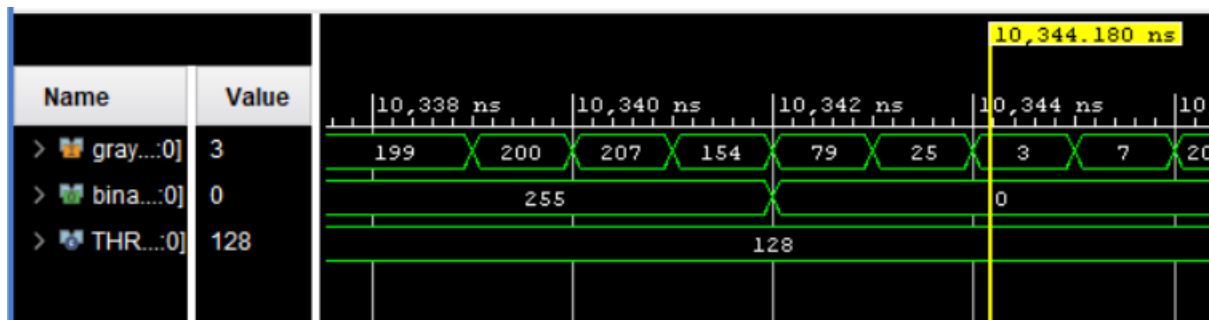
Output:



Grayscale image



Binary output



Result

The Verilog image thresholding system was successfully simulated. The 256×256 grayscale image was read from a text file, processed pixel-by-pixel by the UUT, and converted into a binary image. The output text file contained only two intensity levels, 0 and 255, confirming correct gray-to-binary conversion. The reconstructed output image clearly showed foreground and background separation, verifying the successful implementation of image thresholding using Verilog.